

From idea to *executor*.

The practical playbook for turning ideas into execution using AI.

- 01 Think like an operator
- 02 Brief well and build fast
- 03 Ship something real

From idea to executor.

A practical, founder-led playbook for turning ideas into execution using AI.

Brandon / Aktico / aktico.co

Built by Brandon, co-founder of Surreal. \$500M+ in bookings, 28,000+ contractors coordinated and counting.

Chapter 1

What Actually Changed

Ten years ago I was working three part-time jobs, pitching a business idea to anyone who would listen, and trying to figure out how to close the gap between the concept in my head and something real in the world. The gap felt enormous. Not because the idea was bad. The execution required things I didn't have: technical skills, design skills, money, time, and enough structured thinking to turn a half-formed idea into a coherent plan.

Most ideas die in that gap. Not because the ideas weren't worth pursuing. The perceived distance between "I think I'm onto something" and "here's the thing I built" used to be measured in months, money, and skills you either had or didn't. That perception stopped most people before they started.

That gap is gone.

What this playbook is and isn't

People make more money selling the dream than chasing it. This isn't that. There's no promise of a million dollars, no "quit your job by Friday," no highlight reel of outcomes you're statistically unlikely to reach.

What this is: a practical, founder-led playbook for turning ideas into execution using AI. I'm a co-founder of Surreal, a live entertainment booking and payments platform that has processed more than \$500M in bookings across 28,000+ contractors. The frameworks here come from building it.

You'll learn by doing, not just reading. Every chapter ends with something to do, most of it in 15 minutes. By Chapter 6, you'll have something live on the internet that you built. By Chapter 8, you'll have improved it and shipped it again.

You'll hear the term "AI agents" a lot. An agent is AI that takes actions for you, like sending an email or updating a file, rather than just answering questions. This playbook doesn't use agents yet. It starts with the skill every agent depends on: directing AI clearly. Agents come later in the series.

What is guaranteed: you will learn, and you will learn fast.

The gap is gone

One of the biggest reasons people don't act on ideas isn't laziness. It's the paralysis of not knowing where to begin. The blank page. The feeling that before you can do anything, you need to have everything figured out: the plan, the skills, the resources, the next ten steps laid out cleanly before you take the first one.

You don't. Not anymore. You now have access to something that functions as a genuine extension of your brain. It's available in real time, costs almost nothing, and needs zero structured thinking before you start. Throw the half-formed thought at it. The structure comes out of the conversation, not before it.

Momentum has always been everything at the idea stage. The founders who move fastest aren't the ones with the most resources or the clearest vision on day one. They're the ones who maintain pace, who keep chipping away, learning through doing and adjusting as they go. The tools now mean you can maintain that momentum around the clock.

What this actually looks like

Here's a real example. A friend needed a simple website for his small landscaping business. No developer, no designer, no budget. The brief went in:

EXAMPLE PROMPT

"I need a simple one-page website for a landscaping business based in Melbourne. The owner's name is Dave. His customers are homeowners in the suburbs who want reliable, tidy work without having to manage it themselves. Tone: trustworthy and straightforward, no corporate language. The site needs a one-liner, a short about section, a services list, and a contact form. No stock photos that look nothing like Australian suburbs. Nothing that sounds like a marketing agency wrote it."

What came back was a complete first draft: homepage copy, about section, services list and a suggested page structure. Not perfect, nothing ever is on the first pass, but 80% of the way there in under three minutes. The back-and-forth to close the gap took another twenty. The site was live the same afternoon.

That's not a productivity story. It's a capability story. The skill gap didn't close because anyone learned new skills. It closed because the tools changed.

You don't need to know the tools

This playbook names a few tools along the way. You don't need to know any of them before you start. Every time one is new to you, ask AI what it is before you go any further.

NEVER HEARD OF IT? USE THIS ANY TIME A TOOL OR TERM IS NEW TO YOU

"I've never heard of [tool or term]. Explain what it is, what people use it for, and whether it costs anything. Assume I've never built anything before. Then tell me the one thing I need to know to get started."

Not knowing a tool is never a reason to stop. It's a reason to ask.

When you get stuck

You will hit hurdles. A button isn't where you expected, a tool asks for a setting you don't understand, or something doesn't work the first time. This is where AI is most useful.

Describe the problem to Claude, and paste in any error message or a screenshot. If the answer isn't clear, say so and ask it to explain in simpler terms. Keep going one step at a time until it's resolved.

15-MINUTE EXERCISE

Pick one thing that's been sitting on your list because it felt like too much effort to start. A message you've been avoiding. A plan you haven't mapped out. An idea you've never written down.

Put it into Claude (claude.ai) with as much context as you can. See what comes back.

Don't judge the output yet. Notice how far it gets, where it falls short, and what you'd need to add to close the gap. That observation is your starting point.

Why Most People Use It Badly (And How To Use It Well)

There's a pattern that plays out almost every time someone tries AI for the first time and walks away unimpressed. They type something in, get something generic back, decide it's overhyped, and don't return. The tool didn't fail them. Their mental model did.

The way most people use AI is the way most people use Google. A question goes in, an answer comes out, the exchange is over. That works fine for finding a restaurant or checking a fact. It produces mediocre results for anything that actually matters.

The operator frame

Not every interaction with AI needs to be directed. Some tasks are genuinely just queries: a quick fact check, or a simple draft where the first output is good enough. For those, treating it like a smarter Google is fine. The query mode works.

The operator frame kicks in when the output actually matters. When you're building something real, making a decision with downstream consequences, or trying to get to a result that a generic first draft won't deliver.

When you're in operator mode, stop thinking about AI as something you query. Start thinking about it as something you direct. You define what good looks like. You push back when the first draft isn't there yet. You add context when the output misses the mark. You stay in the loop until the job is done.

Instructions, context, feedback

Three inputs. That's the whole operating mechanic.

Instructions are what you want done. Most people stop here, and it's why most people get generic outputs.

Context is everything the system needs to do the job properly. Who this is for, what the constraints are, what tone is right, what done looks like.

Feedback is what closes the loop. The first output is a draft, not a verdict. React to it. Tell it what landed and what didn't.

WITHOUT CONTEXT (WEAK)

"Write an about page for my business."

WITH CONTEXT (STRONG)

"Write an about page for a small bookkeeping business based in Brisbane. The founder is a former big-four accountant who left to work with small hospitality businesses. Her clients are owners who find the financial side stressful. Tone: direct and warm, like a trusted advisor. No jargon. No corporate language."

Don't let momentum kill quality

The urge to take something close enough and move on is one of the most common ways people short-change themselves. The output that would have actually served you is usually one more round of feedback away. Slowing down inside a conversation to push further is not inefficiency. It's where the real output lives.

How to instruct AI to be honest with you

By default, AI systems are trained to be helpful, which in practice sometimes means agreeable. You can change this with one instruction:

HONESTY INSTRUCTION - ADD TO ANY IMPORTANT CONVERSATION

"When you're not sure about something, say so directly rather than guessing. If you're inferring rather than stating a known fact, flag it. If my question or premise is off, tell me before you answer it. A direct 'I don't know' is more useful to me than a confident-sounding answer that isn't grounded. Push back when you think I'm wrong."

15-MINUTE EXERCISE

Take the task you tried at the end of Chapter 1. Run it again with all three inputs: clear instructions, full context, and the honesty instruction above at the start.

Then react to the first output specifically. Pick one thing that missed and tell it exactly why and what you'd want instead.

Notice the difference between that output and the first one. That difference is the skill this book is building.

Thinking Like an Operator

There's a difference between someone who uses a tool and someone who deploys one. The first person picks it up when they need it and puts it down when they're done. The second person understands what it's capable of, knows when to reach for it, and gets more out of it every time they use it.

What an operator actually does

An operator doesn't start with a tool. They start with an outcome. What needs to exist at the end of this that doesn't exist right now? What does done actually look like? What are the constraints, the non-negotiables, the things that would make the output useless even if it technically answered the brief?

Say you're trying to figure out whether to leave your job and back yourself on an idea you've been sitting on. The non-operator version: "Should I quit my job to start a business?" What comes back will be balanced, reasonable, and completely useless.

OPERATOR BRIEF EXAMPLE

"I'm two years into a stable job, I have six months of savings, a partner with steady income, and an idea I've been testing on the side for eight months with three paying customers. I need a framework for deciding whether now is the right time to back this full time. I don't want generic advice about risk tolerance. I want the specific questions I should be able to answer yes to before I make this call, and an honest read on what's missing from my situation right now."

Before any conversation that matters, answer three questions: What is the goal? What are the constraints? What does good look like?

Your operating system

One of the highest-leverage things you can do early is set up a standing context for the work you do most. You write it once and use it in every conversation where it's relevant. Think of it like the settings on your phone: you configure it once, and everything that runs on top of it benefits.

OPERATING SYSTEM TEMPLATE

"Here's context for our work together. I'm [name]. I'm building [what it is] for [who it's for]. We're at [stage]. My main constraints right now are [constraints]. The voice I write in is [description]. What I'm focused on for the next [timeframe] is [focus]. When I ask you to help with something, assume this context unless I tell you otherwise. If something I ask conflicts with this context, flag it before answering."

You don't have to paste this into every chat. In Claude, you can save it once: add it to your profile preferences in Settings and it applies to every conversation, or add it to a Project's instructions so it applies to one area of work. If you can't find either, ask Claude where they are. v3 of this series covers setting your tools up properly.

15-MINUTE EXERCISE

Write your operating system context using the template above. Keep it to a paragraph, and save it in Claude.

If you have an idea: write the operator brief for one real decision you're sitting on right now.

If you don't have an idea yet: write the brief for a decision someone you know is sitting on. You're practising the discipline, not solving the problem.

Finding Your Project

Everything up to this point has been preparation. This chapter is where the work starts.

Founders learn through doing. Not through consuming more content, not through getting everything perfectly understood before they start, not through waiting until they feel ready. Through doing. This is also not about making money. Not yet. The project you're about to build is a proof of concept for your own capability.

Option A - Build a website for someone else

Look around at the people you know. A family member running a small business with no website. A friend doing something they're good at with no online presence. A local tradie, a personal trainer, a photographer, a baker. Someone whose work is real but whose digital presence either doesn't exist or looks like it was built in 2009.

Use the intake form below. Send it, get the answers back, and you have everything you need to write the brief.

INTAKE FORM - COPY AND SEND

"Hey, I'm working on a project and I'd love to build you a simple website as part of it, at no cost. Just something clean and useful you can actually use. Would you be up for it? If so, can you answer these quickly:

- 1. What's the name of your business or what do you do?*
- 2. Describe what you do in one sentence.*
- 3. Who is your main customer or client?*
- 4. What do you want people to do when they land on your site?*
- 5. Three words that describe the vibe you're going for.*
- 6. Any sites you like the look of?*
- 7. Anything you definitely don't want?*
- 8. Is there anything already online about your business? Links appreciated."*

Option B - Build for your own idea

Have something you've been sitting on? A concept, a side project, a tool, a game? A game is a particularly strong option. The build process is the same, but the output is something people interact with and play. Games

get shared. When someone finishes your game and sends it to a friend, that's real validation without any marketing required.

Writing the brief

Use the three questions from Chapter 3 as your structure.

EXAMPLE BRIEF - OPTION A

"I'm building a one-page website for a Melbourne-based landscaping business. The owner's name is Dave. His customers are time-poor homeowners who want reliable, tidy work. The site needs: a one-liner, a short about section, a services list, and a contact form. Tone: straightforward and trustworthy. No stock photography. Nothing that sounds like a chain. Here are links to his existing online presence: [Facebook page] [Google listing]."

15-MINUTE EXERCISE

Option A: send the intake form to one person today. When the answers come back, write your brief using the example above.

Option B: write your game concept in one plain paragraph. What it is, what the player does, and what finishing it looks like. Then write the brief using the three questions from Chapter 3.

Chapter 5

Building It

This is where the brief becomes a real thing. It will feel messier than the previous chapters. That's correct. Building is messier than planning, and the mess is where the learning happens.

Option A - Building a website

Step 1: Generate the content

Before you touch any build tool, get the words right. Open Claude at claude.ai. Make sure your operating context is saved or pasted in, then send your brief with this added:

COPY GENERATION PROMPT

"Based on this brief, write all the copy for a one-page website. Include: a headline, a one-line subheading, an about section of no more than three sentences, a services list with short descriptions, and a call to action. Write it as if a real person wrote it, not a marketing agency. Give me two headline options."

Step 2: Design intent

DESIGN PROMPT

"Based on the brief and copy we've written, describe the visual direction for this website. Suggest a colour palette with hex codes (e.g. #1A1A1A). Suggest font styles. Describe the layout top to bottom. Keep it simple: a one-page site for a small business."

Step 3: Build in Carrd

For a first website with no prior experience, use Carrd (carrd.co). It's a drag-and-drop website builder: you build a one-page site by moving pieces around on screen, without writing any code. New to Carrd? Use the never-heard-of-it prompt from Chapter 1.

Carrd is free to start, and the free plan publishes your site at a carrd.co address. Two things need the paid Pro Standard plan, roughly \$19 USD a year at the time of writing: a contact form, and your own domain (more on domains in Chapter 7). On the free plan, use a button that links to an email address or phone number instead of a form. Check Carrd's current pricing before you pay for anything.

If you get stuck, ask Claude: "I'm building a website in Carrd. Here's what I'm trying to do: [describe it]. Walk me through it in simple steps."

Option B - Building an interactive game

Step 1: Define the concept

Start from the concept paragraph you wrote in Chapter 4. Simple mechanics work best: a quiz, a reaction game, a word puzzle, a choose-your-own-adventure.

Step 2: Generate the game

GAME GENERATION PROMPT

"I want to design a simple browser-based interactive game. A browser-based game runs directly in Chrome or Safari with no downloads required. Here's the concept: [describe it]. Help me design the game before we build. Define: the core mechanic, the win condition, the basic layout, and the content needed. Keep it simple enough for a first-time builder. Ask me any questions before we design."

Once designed, follow with:

"Now build this as a single HTML file with CSS and JavaScript included. It should run entirely in a browser. Make it interactive: the player should actually be able to play it. Mobile friendly. Give me the complete file in one block."

An HTML file is the type of file web pages are made from. CSS controls how it looks and JavaScript makes it interactive. You don't need to understand either. Claude puts all three into one file, and your whole game lives inside it.

Step 3: Put it online with Netlify Drop

Netlify (netlify.com) is a free service that puts files on the internet. Its Netlify Drop feature lets you do it by dragging a folder into your web browser, with no code or other tools needed. Do this step on a computer, as drag and drop doesn't work on phones.

1. Save the code Claude gave you as a file called **index.html**. The name matters: it tells the browser this is the main page. Not sure how? Ask Claude: "How do I save this code as a file called index.html on my [Mac / Windows] computer?"
2. Put that file in a new folder on your computer, on its own.
3. Go to app.netlify.com/drop and drag the folder onto the page.
4. Create a free account when prompted, so your site is kept.
5. Netlify gives you a URL (a web address) ending in netlify.app. That's your game, live.

If anything doesn't go to plan, ask Claude: "I'm putting an HTML file online with Netlify Drop. I have no technical experience. Here's what's happening: [describe it]. What should I do next?"

THE BUILD

Ship the first draft. Not perfect. Live.

The goal of this step is a URL that works, not a URL that's finished.

Chapter 6

Shipping It

This is the shortest chapter in the book. It's also the most important one.

Everything before this point has been in service of one moment: putting something you built in front of a real person and watching them react to it. That moment is what separates people who learn about building things from people who actually build them. You are about to be the second kind of person.

Send the link

For Option A: send it to the person you built it for. Not a screenshot. The live link. Let them open it on their phone as a stranger would. Don't explain it before they see it. Don't pre-empt the parts you think aren't finished. Resist it. Send the link. Let them land on it cold.

For Option B: share with three or four people. Send the link with one line of context: what it is and that you built it. Nothing more.

Collecting feedback

When they've seen it, ask three questions:

FEEDBACK QUESTIONS

"What's the first thing you noticed?"

"Was there anything confusing or unclear?"

"Is there anything you expected to see that wasn't there?"

15-MINUTE EXERCISE

Send your link today. Ask the three questions above and write down the answers word for word, not your summary of them. You'll use them in Chapter 8.

You built something. It's live. Someone else has seen it. The next two chapters make the address your own and turn that feedback into your first update.

Make the Address Your Own

Right now your project lives at an address someone else chose, something like `brave-otter-12ab34.netlify.app` or a `carrd.co` address. That's fine for a first ship. When you want something cleaner, you have two options: a free one and a paid one.

What a domain is

A domain is the name part of a web address, like `aktico.co`. You don't own it outright. You rent it by the year from a company called a domain registrar, and you can renew it each year.

Your project already has an address that includes someone else's domain: `netlify.app` or `carrd.co`. The free option changes the part in front of it. The paid option gives you a domain of your own.

Option 1: Rename your free address (free)

This gets you a clean, readable address like `daves-landscaping.netlify.app` at no cost.

On Netlify: open your site's page in Netlify, find the Production domains panel, and choose Edit site name from the options next to your current address. Type a new name and save. The old random address stops working once you rename it, so share the new one.

On Carrd: the address is chosen when you publish and can be changed in your site's settings. If you can't find it, ask Claude to walk you through it.

For most first projects, this is all you need.

Option 2: Use your own domain (paid)

This gets you an address like `davelandscaping.com.au`. It costs money every year, so it's worth it once the project matters to you or to the person you built it for.

1. **Buy the domain** from a registrar. Prices vary, so compare a few before you buy. Note that Australian `.com.au` domains need an Australian business connection, such as an ABN.
2. **Connect it to your site.** On Netlify, select Add a domain at the bottom of the Production domains panel. Netlify's free plan supports this. On Carrd, you need the Pro Standard plan.

3. **Update the domain's settings** at your registrar. These are called DNS settings: think of DNS as the internet's address book, telling browsers where to find your site. Netlify or Carrd will tell you exactly what to enter.
4. **Wait.** DNS changes can take a few hours to work. If the address doesn't load straight away, that's normal.

DOMAIN SETUP PROMPT

"I've bought the domain [your domain] from [registrar] and want it to show my site, which is hosted on [Netlify / Carrd]. I've never done this before. Walk me through connecting it one step at a time, and tell me what to expect while it updates."

15-MINUTE EXERCISE

Rename your free address to something short and readable, and open it on your phone to check it works. Then decide whether your own domain is worth paying for yet. For most first projects, the honest answer is not yet.

Ship It Again

Shipping once is the start, not the finish. The people who get good at building aren't the ones whose first version was great. They're the ones who shipped the second, third and fourth versions.

The loop

Every improvement follows the same five steps:

1. **Read the feedback** you wrote down in Chapter 6.
2. **Pick one change.** Not five. The one that would make the biggest difference.
3. **Make it** with Claude.
4. **Test it** on your phone and a computer before anyone else sees it.
5. **Re-ship it**, then tell the people who gave feedback what you changed.

ITERATION PROMPT

"I built a [website / game] and shared it with people. Here is the feedback I received: [list feedback]. Based on this, tell me the three most important changes I should make and why. Then help me make them one at a time."

Making the change

Option A, Carrd: make the change in the Carrd editor and publish again. Your live site updates at the same address.

Option B, your game: paste your current file into Claude and ask for the change.

CHANGE PROMPT

"Here's the current version of my game: [paste the full file]. Make this one change: [describe it]. Don't change anything else. Give me the complete updated file in one block."

Keep your last working version

Before you replace anything, save a copy of the version that works. Make a folder called versions and copy your current index.html into it, renamed with a number, like index-v1.html. If the new version breaks, you can put the old one back in minutes.

Re-shipping with Netlify Drop

Save the updated code over index.html in your site folder. Then open your site's page in Netlify, and drag the folder onto the drag-and-drop area under Production deploys. Your site updates at the same address.

Don't drag it onto the original Netlify Drop page again. That creates a separate new site with a new address.

New content and features

Adding something new works the same way: one change at a time, tested, then shipped. A new level, a new page, a new section. If you try to add five things at once and something breaks, you won't know which one caused it.

15-MINUTE EXERCISE

Pick the single most useful change from your Chapter 6 feedback. Make it, test it on your phone, and ship it. Then message the person who suggested it and tell them it's live.

What you've just done is close the gap. The gap between idea and shipped thing that stops most people before they start, you've closed it. Then you closed it again. With tools that cost almost nothing, in a timeframe that would have been impossible two years ago.

Founders talk about their first ship for years. Not because the thing they built was extraordinary. Because the feeling of putting something real into the world for the first time is one that stays with you. It recalibrates what you think is possible.

You built something. It's live. Someone else has seen it. And you've made it better.

This is the foundation. In v2, you'll build your first repeatable workflow: a set of steps you design once and run again, with AI drafting the content and a free scheduling tool publishing it at the time you choose. You set it up once, check what it produces, and it keeps going on schedule while you're doing something else.

That's a different level entirely. We'll see you there.